

Phase 34: Finite Partial Functions

Finite partial functions are partial functions with a finite domain. The operator $\dashv\vdash$ denotes a finite partial function, meaning the function may not be defined everywhere, and its domain must be finite.

Example 1 : Basic Finite Function Type

A finite partial function from X to Y is a partial function whose domain is finite. This is useful when modeling resources with limited capacity or databases with a bounded number of entries.

$[Year, TableFF]$

$records : Year \dashv\vdash TableFF$
$\#(\text{dom } records) \leq 1000$

The records relation is a finite partial function because: 1. It is partial: not all years need to have a table 2. Its domain is finite: at most 1000 years have records 3. It is functional: each year maps to at most one table

Example 2 : Comparison with Other Function Types

Comparison of function type operators:

- Total function ($\rightarrow$): defined for all inputs, domain is entire source set - Partial function ($\leftrightarrow$): may not be defined for all inputs, domain may be infinite - Finite partial function ($\dashv\vdash$): partial function with finite domain - Total bijection ($\succ\!\!\rightarrow$): defined everywhere, injective and surjective

$[X, Y]$

$totalFunc : X \rightarrow Y$ $partialFunc : X \leftrightarrow Y$ $finitePartialFunc : X \dashv\vdash Y$ $totalBij : X \succ\!\!\rightarrow Y$
$\text{dom } totalFunc = X \wedge \text{dom } totalBij = X \wedge (\forall x1, x2 : X \mid totalBij(x1) = totalBij(x2) \bullet x1 = x2) \wedge \text{ran } totalBij = Y$

Example 3 : Finite Functions elem Practice

Real-world example: A database table storing customer preferences.

$[CustomerID, Preference]$

$preferences : CustomerID \dashv\vdash Preference$
$\#(\text{dom } preferences) \leq 10000$

This models a preference database where: - Not all customers have preferences recorded (partial) - The database has a capacity limit (finite domain) - Each customer maps to one preference value (functional)

Example 4 : Operations on Finite Functions

$$\begin{array}{|l}
 f : \mathbb{N} \multimap \mathbb{N} \\
 g : \mathbb{N} \multimap \mathbb{N} \\
 \hline
 f = \{1 \mapsto 10, 2 \mapsto 20, 3 \mapsto 30\} \wedge \\
 g = \{10 \mapsto 100, 20 \mapsto 200\} \wedge \\
 \#(\text{dom } f) = 3 \wedge \\
 \#(\text{dom } g) = 2
 \end{array}$$

We can compose finite functions, but the result may not be finite unless proven.

$$\begin{array}{|l}
 f_composed : \mathbb{N} \multimap \mathbb{N} \\
 \hline
 f_composed = g \circ f \wedge \#(\text{dom } f_composed) \leq \#(\text{dom } f)
 \end{array}$$

The composition has a domain size bounded by the domain of f , since we can only compose where f 's range intersects g 's domain.

Example 5 : Domain Restrictions land Finiteness

$[Title, Length, ViewDate]$

$$\begin{array}{|l}
 viewed : Title \multimap ViewDate \\
 recent_viewed : Title \multimap ViewDate \\
 recentSet : \mathbb{P} \text{ Title} \\
 \hline
 recentSet = \{ t : Title \mid t \in \text{dom } viewed \} \wedge \\
 recent_viewed = recentSet \triangleleft viewed \wedge \\
 \#(\text{dom } recent_viewed) \leq \#(\text{dom } viewed)
 \end{array}$$

Restricting a finite partial function preserves finiteness. The `recent_viewed` function has a domain that is a subset of `viewed`'s domain, so it remains finite.